Jimcan Operations Dashboard: Technical Requirements Document

Version 0.1 (draft for review) | 24 October 2026 | Author: George | Companion to the PRD v0.1

1. Purpose

This TRD defines **how** the system in the PRD is built, hosted, secured, tested, deployed and kept running. It turns requirements into technical decisions, constraints and checks. Where a decision depends on facts not yet verified (mainly hosting limits), it is marked **VERIFY** and tied to a spike in Section 17.

Guiding principle: the simplest thing that is correct, secure and recoverable. A boring, well tested Django monolith beats a clever architecture for a small firm with one developer.

2. Technical constraints

Constraint	Consequence
No paid services except domain and Namecheap hosting	Only free, open source tools. No managed database, queue, storage, email or monitoring subscriptions
Shared hosting (cPanel, CloudLinux, Passenger)	No root. Limited memory and processes. Python via cPanel Setup Python App. No Docker, no Redis, no long running workers
Weak connectivity on site	Small pages, compressed uploads, retry queue, no heavy frameworks
Single developer	Few moving parts, strong tests, written runbooks
Money and tax data	Exact decimals, audit history, tested backups, strict access control
Uganda context	UGX, East Africa Time, URA tax rules as editable settings

3. Architecture overview

One Django application, server rendered, with a relational database, private file storage and cron driven jobs.

Request path: Browser, then Namecheap Apache with Passenger, then Django (WhiteNoise serves static files), then a service layer, then the database. Pages return HTML. HTMX swaps fragments. Alpine handles small browser behaviour. Chart.js draws charts from JSON embedded in the page.

Components

Component	Role
Django app (monolith)	Pages, forms, business logic, admin, public lead endpoint
Database	MySQL from cPanel in production. SQLite locally
Private media store	Receipts, photos, drawings, generated PDFs, outside the web root
Cron jobs	Backups, reminders, overdue flagging, email outbox, health pings
Email	Namecheap SMTP through a database outbox
Static public website	jimcanconstruction.site posts enquiries to the Django lead endpoint
Free external helpers	GitHub (code and CI), UptimeRobot or Healthchecks.io (monitoring), a free cloud drive for off server backup copies

No microservices, no separate API for the UI, no single page app, no message broker.

4. Technology decisions

Area	Decision	Why	Rejected
Language and framework	Python and Django 5.2 LTS (confirm host Python is 3.10 or higher)	You know it, batteries included, strong security defaults, built in admin and auth	Next.js, Supabase, Vercel: paid at scale, extra complexity, outside your stack

Area	Decision	Why	Rejected
Database	MySQL (InnoDB) in production, SQLite in development and tests	Free in cPanel, transactional, row locking for numbering	PostgreSQL if host offers it free (preferred if available, VERIFY)
DB driver	PyMySQL, or mysqlclient if it installs on the host	PyMySQL is pure Python and installs anywhere	None
Templates and UI	Django templates, Tailwind CSS compiled locally, HTMX 2, Alpine.js 3, Chart.js 4	No build server needed in production, small payloads	React or Vue: second codebase, API work
Static files	WhiteNoise	Works with Passenger without extra Apache config	Separate CDN
History and audit	django-simple-history for money tables, plus a custom AuditEvent for status changes and voids	Free, proven, field level diffs	Hand written triggers
Login protection	django-axes	Throttling and lockout	Custom code
2FA (should have)	django-otp with TOTP	Free, standards based	SMS codes
Filtering	django-filter	Fast list screens	Hand written filters
PDF	ReportLab (pure Python). Spike to test xhtml2pdf as an HTML based alternative	WeasyPrint needs system libraries that shared hosting may not offer	WeasyPrint unless the spike shows it runs
Excel	openpyxl	Pure Python, can write formulas and styles	pandas for exports
Images	Pillow	Resize, strip EXIF, validate	None
Testing	Django test runner, coverage.py, Playwright for 3 smoke journeys	Free, standard	Large E2E suite

Area	Decision	Why	Rejected
CI	GitHub Actions free tier	Free for small private repos	Paid CI

Rule: every dependency must be free, actively maintained, and pinned to an exact version in requirements.txt. Fewer is better.

5. Hosting and environment

5.1 Verify before building (spike, week 1)

Deploy a hello world Django app to the actual Namecheap plan and confirm:

1. Python version available and Django 5.2 installs
2. MySQL available, connection works, character set utf8mb4
3. PyMySQL or mysqlclient installs in the app virtualenv
4. ReportLab, Pillow and openpyxl install and run
5. Cron jobs allowed, minimum interval, and a management command runs within time limits
6. SSH access, or cPanel Git deployment as fallback
7. SMTP sending limits per hour and per day
8. Memory, process and file count limits (CloudLinux LVE) and what happens at the limit
9. SSL certificate (AutoSSL) and forced HTTPS
10. Disk quota against expected media growth
11. Ability to keep a directory outside public_html for private media and backups

If any item fails, the plan changes before a single feature is built. The fallback is a small VPS, which the same code supports.

5.2 Environments

Environment	Purpose	Data
Local	Development	SQLite, fake data
Staging (a second cPanel app or subdomain)	Pre release checks, UAT	Copy of anonymised data
Production	Live use	Real data

5.3 Configuration and secrets

- All settings from environment variables set in cPanel. No secrets in git.
- Required variables: secret key, debug flag, allowed hosts, CSRF origins, database credentials, SMTP credentials, admin alert email, media path, backup path.
- Production defaults: debug off, secure cookies, HTTPS redirect, HSTS enabled after HTTPS is proven stable.
- Separate secret key per environment. Rotation procedure documented.

6. Application structure

```
config/        settings (base, local, production), urls, wsgi
core/          base models, soft delete, money helpers, numbering, audit,
settings store
accounts/      users, roles, permissions, login
crm/           clients, contacts, leads, activities, public lead intake
projects/      projects, phases, phase history, milestones
boq/           rate library, BOQs, versions, sections, items, calculation
engine, exports
sales/         quotes, quote templates
finance/       invoices, payments, expenses, suppliers, categories
tax/           tax rates, tax periods, filings, summaries
reports/       statements, dashboard queries, exports
files/         uploads, validation, private serving
notify/        notifications, email outbox
templates/     base, partials, components
static/        compiled CSS, vendored JS, fonts
```

Layering rule (keeps logic testable):

1. **Models:** data and simple constraints only
2. **Services:** all business rules and writes (for example, `boq.services.lock_version`, `finance.services.record_payment`). Always in database transactions
3. **Selectors:** read queries and report calculations
4. **Views:** permissions, forms, calling services, rendering. No business arithmetic in views or templates
5. **Templates:** display only, using shared components

7. Data and money handling

- All money fields use `DecimalField`. Floats are banned in money code and a test greps for them.

- Amount fields: max 16 digits, 2 decimal places. Quantities: 3 decimal places. Rates: 2 decimal places. Percentages: 2 decimal places.
- One helper, `quantize_ugx()`, rounds half up to whole shillings and is the only place rounding happens.
- Every money record carries a `currency` field (default UGX) so USD can be added later without a schema change. MVP forbids mixing currencies inside one document.
- Stored totals on issued documents are snapshots. They never recalculate after issue.
- Database constraints back up code rules: non negative quantities and amounts, valid status values, unique numbers, foreign keys with protective delete rules.
- All timestamps stored in UTC and shown in East Africa Time.
- Character set utf8mb4 so names and symbols store correctly.

8. Security design

8.1 Authentication

- Django auth with strong password validators (minimum 10 characters, common password check).
- Login throttling and lockout through django-axes. Lockouts are logged.
- Session timeout after inactivity (default 8 hours, Admin sessions shorter). Session cookies are secure, HTTP only, SameSite Lax.
- Admin accounts use TOTP 2FA once django-otp is added (should have, plan for it now).
- No public registration. Users are created by Admin only.

8.2 Authorisation (default deny)

- Django's `LoginRequiredMiddleware` forces login for every view. Public views, such as the lead endpoint, are explicitly opted out.
- Roles are Django Groups with named permissions: Admin, Manager, Site Engineer, Accountant.
- A `RoleRequiredMixin` and decorator guard each view. A test walks the full URL list and fails if any view lacks a guard.
- **Object level scoping:** each model exposes `for_user(user)` returning only records the user may see. Site Engineers get only assigned projects and their files. Views never use unscoped querysets.
- Pricing, margins and statements are hidden from Site Engineers at the query level, not just by hiding buttons.

- The built in Django admin moves to a non obvious URL, is restricted to the Admin role, and is used for master data and settings only, never for money edits that bypass services.

8.3 Application protections

- CSRF on all forms. HTMX sends the CSRF header on every request.
- Output escaping on by default. No `|safe` on user input.
- ORM queries only. No raw SQL with string formatting.
- Content Security Policy via django-csp. All fonts and scripts are self hosted. Alpine's standard build needs `unsafe-eval`, so MVP allows it for scripts only and the CSP build of Alpine is a hardening task.
- Security headers: HSTS, no sniffing, frame denial, strict referrer policy.
- Clickjacking protection on.
- Rate limiting on login, lead intake and file upload.
- Error pages never show stack traces in production.

8.4 File security

- Media stored outside the web root. Downloads go through a Django view that checks project access.
- Validation: extension allow list, content sniffing, size cap (8 MB server side), image verification with Pillow, EXIF stripped, filenames replaced with random names.
- PDFs and drawings are served as downloads, not rendered inline from untrusted input.
- No executable file types.

8.5 Public lead endpoint

The static site posts to `/public/leads/`:

- Accepts only name, phone, email, project type, location, details.
- Honeypot field and a signed timestamp (rejects forms submitted in under 3 seconds or older than 2 hours).
- IP based rate limit stored in the database (for example 5 per hour).
- Cloudflare Turnstile (free) optional if spam appears.
- CORS allowed only for the Jimcan site origin. Endpoint is CSRF exempt but protected by the signed token and limits.
- Input length limits and validation. Result is a lead row and an email outbox entry.

8.6 Data protection

- Data minimisation and retention rules from the PRD applied in code (soft delete, archive, export).
- Personal data never written to logs.
- Backups are stored in a private location, and any copy off the server is encrypted.
- Staff access reviewed at least quarterly.

9. Audit and history

- django-simple-history on leads, clients, projects, BOQs, quotes, invoices, payments, expenses, tax periods and settings.
- A custom `AuditEvent` records business events: lead status change, phase change, lock, issue, void, payment, filing, user changes, permission denials, override use.
- Audit tables expose no edit or delete path in the app. Database user for the app has no need for DDL in production.
- Soft delete through a base model with `is_deleted`, `deleted_at`, `deleted_by`. The default manager hides deleted rows. Only Admin can restore.
- Hard deletion is blocked for financial models by overriding delete and by tests.

10. Key technical designs

10.1 BOQ engine

- A pure Python module `boq/calc.py` takes sections, items and settings and returns every line amount and total using the PRD order. No database access inside it, so it is easy to test.
- The browser shows live totals using the same formulas in a small JavaScript function. **The server result is authoritative.** A shared set of test vectors (JSON) runs against both Python and JavaScript in the test suite, so they cannot drift.
- Versions: `BOQ` has many `BOQVersion`. Items belong to a version. A version has status Draft, Locked. Locking copies computed totals into snapshot fields and sets a hash of the content.
- Editing a locked version is rejected in the service layer and by a database level guard.
- Rate library changes never touch locked versions. Draft versions can refresh from the library on user request only, and items store the rate used.
- Editing grid: HTMX row updates with debounce, saving each change as a small request. Bulk paste handled by a dedicated endpoint with validation preview.

- Target: 300 line BOQ renders under 1 second server side and saves a row edit under 300 ms.

10.2 Document numbering

- Table `DocumentSequence(key, year, last_value)`.
- Allocation runs inside `transaction.atomic()` with `select_for_update()`, then increments, then saves the document in the same transaction. A failed save rolls back the number, so no gaps.
- Format configurable, for example `INV-2026-0001`.
- Unique constraint on number. Voiding keeps the number.
- Tested with concurrent requests.

10.3 Invoices, payments, withholding tax

- Payments and WHT entries are separate rows linked to an invoice. Balance is computed by a single service function and cached on the invoice, then verified by a nightly consistency check.
- Status transitions follow a state table in code. Illegal transitions raise errors.
- Overdue flag set by the daily job and also computed on read, so a missed job never shows wrong data.

10.4 Tax

- `TaxRate(type, rate, effective_from, effective_to)` with the active rate chosen by document date.
- Documents store the rate they used.
- `TaxPeriod(month, status, filed_on, reference)`. Filing sets status Filed. Writes into a filed period go through an Admin override service that logs a reason.
- VAT summary computed by a selector from non void invoices and recorded purchase VAT in the period, with drill down to the source rows.

10.5 Statements and reporting

- MVP derives statements from source tables through tested selector functions, not a separate ledger. This is simpler and fast enough for the target volumes.
- Every report declares its basis (cash or accrual) and period.
- A reconciliation page recomputes totals two independent ways and shows any difference. A failing reconciliation raises an alert.
- Database indexes on date, project, status and category columns used by reports.

- If volumes or accounting needs grow, a `LedgerEntry` table can be introduced behind the same selector interface.

10.6 PDF and Excel

- PDF templates for quote, invoice, BOQ, statements, tax summary built with ReportLab, using embedded fonts, company logo, tax numbers and terms from settings.
- Generated on demand, streamed to the user, and a copy stored for issued quotes and invoices so the issued document never changes.
- Excel exports use openpyxl with formulas for quantity times rate and totals where useful.
- Heavy exports run within request limits (VERIFY). If the host kills long requests, the job runs through the cron queue and the user is notified when ready.

10.7 Uploads and offline tolerance

- Browser compresses photos to about 1 MB (maximum 2000 px on the long side) before sending, using a vendored compression library.
- Each upload has a client generated UUID, so retries never create duplicates.
- A small service worker and IndexedDB queue hold pending uploads. The queue retries on the `online` event, on app open and periodically. Background Sync is used where supported, with a fallback for iOS.
- The Site Engineer screen shows the waiting list and statuses.
- The site is installable (manifest and service worker). Caching is limited to static assets and the upload screen shell. Financial pages are never cached for offline use.

10.8 Notifications and email outbox

- Events write rows to `EmailOutbox` and `Notification`.
- A cron command sends pending emails every 5 minutes, with retry counts and backoff, respecting hourly SMTP limits.
- Templates are plain and mobile friendly. Plain text version always included.
- SPF, DKIM and DMARC for the sending domain are configured so mail reaches inboxes.

10.9 Search

- Simple database search using indexed `icontains` on a few fields and filters. No search server in MVP.

11. Frontend technical requirements

- Tailwind CSS compiled on the developer machine with the standalone CLI. Only the minified CSS file is deployed.
- Design tokens from the Design Brief are defined once in the Tailwind config.
- Shared template components: KPI card, table, chip, progress, form field, empty state, modal, toast.
- HTMX used for filters, inline edits, partial table refresh and modals. Every HTMX route also works as a normal page load.
- Alpine used only for small local state: menu, tabs, dropdowns, live BOQ totals.
- Forms use Django forms and are rendered with shared partials, with errors shown beside fields.
- Accessibility: semantic HTML, labels on all fields, visible focus, colour contrast of at least 4.5 to 1, keyboard operable.
- Browser support: last two versions of Chrome, Edge, Firefox, Safari, and Android and iOS browsers from the last 3 years.
- Budget: each page under 150 KB of HTML, CSS and JS excluding fonts and images, and fonts preloaded once.

12. Performance plan

Target	Approach
Pages under 4 s on 3G	Server rendering, small assets, long cache headers for hashed static files
List screens	Pagination (25 rows), `select_related` and `prefetch_related`, indexes
Reports under 10 s for a year	Aggregation in the database, indexes, date limited queries
BOQ edit under 300 ms	Row level saves, minimal payloads
Memory limits	Few workers, no heavy libraries loaded at startup, paginated exports

Query counts are asserted in tests for the busiest screens to prevent regressions. Django Debug Toolbar is used locally only.

13. Background jobs (cron and management commands)

Job	Schedule	Purpose
Email outbox sender	Every 5 minutes	Send pending emails with retry
Overdue and follow up check	Daily 06:00	Flag overdue invoices and leads, create notifications
Tax reminders	Daily 07:00	Remind before due dates
Database backup	Daily 01:00	Dump, compress, rotate
Media backup	Weekly (daily incrementals if quota allows)	Archive media to backup folder
Consistency checks	Nightly 02:00	Invoice balances, BOQ snapshots, reconciliation, orphaned files
Health ping	Every 15 minutes	Ping an external monitor so a silent failure raises an alert
Cleanup	Weekly	Expire sessions, purge old rate limit rows

Every command is idempotent, logs its result, and reports failures by email. If the host limits cron frequency, the schedule is adjusted (VERIFY).

14. Backup and disaster recovery

Item	Target
Recovery point objective (maximum data loss)	24 hours, improved to a few hours if the host allows more frequent dumps
Recovery time objective	One working day
Retention	14 daily, 8 weekly, 12 monthly database backups
Locations	On server backup folder, plus an off server encrypted copy in a free cloud drive
Off server copy method	Scripted upload where the host allows it, otherwise a monthly manual download plus a weekly email of the compressed database if small

Item	Target
Restore drill	Monthly for the first 3 months, then quarterly. Restore to a clean copy and run the reconciliation page
Runbook	Written steps, tested by someone other than the author at least once

A backup that has never been restored is not counted as a backup.

15. Logging and monitoring

- Application log to a rotating file, with a request ID. Errors also email the Admin address through Django's built in error mail.
- Business events are in the audit tables, not only in logs.
- Free uptime monitoring with an alert to a phone number or email that a person actually reads.
- Healthchecks.io style pings from cron, so a stopped job is noticed.
- Weekly look at logs, disk usage and backup status, written into the runbook.
- Optional free tier of an error tracker (for example Sentry free) may be added if the host allows outbound calls. Not required.

16. Testing strategy

Layer	Scope	Tooling
Unit	BOQ calc, rounding, VAT, WHT, numbering, status transitions, permission rules	Django tests, shared JSON vectors
Integration	Services with the database: lock BOQ, issue invoice, record payment, file filing a period	Django tests
Permission	Every role against every view, object scoping, direct URL guessing	Generated test matrix
Reconciliation invariants	Statement totals equal source totals across randomised data	Property style tests
Concurrency	Parallel numbering and payment recording	Threaded tests (on MySQL in CI)

Layer	Scope	Tooling
Smoke E2E	Login, create BOQ, issue invoice and payment, upload photo	Playwright, 3 journeys
Manual	Phone on weak network, PDF layouts, print, email delivery	Checklist
UAT	Real historic BOQs and invoices replayed	Signed off by Jimcan

Minimum bars: 90% coverage on `calc`, `services`, and tax and finance selectors. All tests pass before any release. CI runs tests on every push using GitHub Actions. Test fixtures include realistic Ugandan construction BOQs, not toy data.

17. Technical spikes (first two weeks)

1. Hosting verification checklist in 5.1
2. PDF generation on the host with ReportLab, with a quote sample
3. Excel export of a 300 line BOQ
4. Email delivery and SPF, DKIM, DMARC check
5. Photo compression and queued upload test on a real phone with throttled network
6. Backup, copy off server, and a full restore
7. Deployment, rollback and restart procedure

Each spike has a pass or fail result recorded in the repo.

18. Deployment and release process

Source control: private GitHub repository, main branch always deployable, short lived feature branches, tagged releases (v0.1.0 and onward), conventional commit messages.

Release steps (production)

1. Tests green in CI
2. Announce a maintenance window if needed
3. Run a fresh database backup and confirm file size
4. Pull the tag on the server (SSH or cPanel Git)
5. Install pinned requirements in the virtualenv
6. Run migrations
7. Collect static files

8. Restart the app by touching the Passenger restart file
9. Run a smoke checklist (login, dashboard, a PDF, an upload)
10. Watch the logs for 15 minutes

Rollback: redeploy the previous tag. If a migration changed data, restore the pre release backup. Migrations follow an expand then contract pattern so the previous version still runs against the new schema.

Hot fixes: a branch from the release tag, tested, tagged, deployed with the same steps.

19. Scalability path

Trigger	Action
Slow pages, memory limits hit	Move to a small VPS, use gunicorn behind Nginx, same code
Many concurrent users	Add caching with the database or local memory cache, then Redis on the VPS
Long running exports	Add a task queue (for example Django Q or Celery) on the VPS
Need for strict accounting	Introduce a ledger table and double entry behind existing selectors
Large media	Move media to object storage
More companies	Introduce a tenant model (a larger change, deliberately not designed now)

20. Technical risks

Risk	Mitigation
Host cannot run chosen PDF library	Early spike, ReportLab or xhtml2pdf fallback, VPS fallback
Request time limits break exports	Cron backed export queue
SMTP hourly limits	Outbox with throttling, digest emails
Shared hosting resource spikes	Efficient queries, pagination, avoid heavy imports

Risk	Mitigation
Calculation drift between browser and server	Shared test vectors, server authoritative
Migration failure in production	Backup before each release, staging rehearsal, expand then contract
Lost developer access	Documented runbooks, credentials stored in a password manager held by Admin
Dependency vulnerabilities	Pinned versions, monthly update review, `pip audit` in CI

21. Technical definition of done

A feature is done when:

1. It meets its PRD acceptance criteria
2. Business logic sits in a service with unit tests
3. Permissions and object scoping are enforced and tested
4. Money uses decimals and goes through the rounding helper
5. Changes are audited where required
6. It works on a phone and at desktop width, with keyboard access
7. Errors and empty states are handled
8. No new query performance regressions
9. Migrations are reversible or safe
10. Documentation or runbook updated if operations changed
11. Deployed to staging and checked

22. Open technical questions

1. Which Namecheap plan and Python version, and is PostgreSQL available?
2. SSH access or cPanel Git only?
3. SMTP limits and mailbox for sending?
4. Maximum request time and memory per process?
5. Which free off server backup location will Jimcan own and control?
6. Who holds production credentials besides the developer?
7. Should the public site stay static or later move into Django?
8. Is a staging environment possible on the same plan?

9. Does EFRIS integration need to be planned for in the invoice design now?

23. Approvals

Name	Role	Decision	Date
George	Developer and technical owner		
	Jimcan owner or director		
	Reviewer (second developer or technical adviser)		